Problema 1

Tomamos los modelos base de las clases, a los cuales añadimos una función evalP, la cual nos permite evaluar la probabilidad de observar una imagen $p(x)$, además de una función de entrenamiento.

Cargamos MNIST y creamos los loaders de entrenamiento y prueba

```
import torch
import torch.nn as nn
import torchvision.transforms as tt
import torch.nn.functional as F
import torch.distributions as td
import numpy as np
import matplotlib.pyplot as plt
from torch.utils.data import Dataset, DataLoader
from torchvision import datasets, transforms


transform=transforms.Compose([
    transforms.ToTensor(),
])
dataset_train = datasets.MNIST('data/', train=True, download=True,transform=transform)
dataset_test = datasets.MNIST('data/', train=False, transform=transform)
training_loader = torch.utils.data.DataLoader(dataset_train,batch_size=1024)
testing_loader = torch.utils.data.DataLoader(dataset_test,batch_size=1024)
```

Iniciamos con la mezcla de Gaussianas

```
class LowerTriangularPositive(nn.Module):

    def __init__(self, K, D):
        super().__init__()
        # Diagonal terms initialized at one
        diags = torch.diag(torch.ones((D))).repeat(K,1,1)
        outdiags = 0.01*torch.randn(K, D, D)
        self.tril = nn.Parameter(diags+outdiags)
        self.register_buffer('mask', torch.tril(torch.ones(D, D), diagonal=-1))
        self.register_buffer('diag_mask', torch.diag(torch.ones(D)))

    def forward(self):
        L = torch.zeros_like(self.tril)
        for i in range(self.tril.shape[0]):
            L[i] = self.tril[i] * self.mask
            L[i] = L[i] + self.diag_mask * torch.sigmoid(self.tril[i].diag()).diag() # Ensure positive diagonal
        return L


class MoG(nn.Module):

    def __init__(self, D, K):
        super(MoG, self).__init__()
        # hyperparams
        self.D = D # the dimensionality of the input
        self.K = K # the number of components
        # params
        self.M = nn.Parameter(torch.sigmoid(torch.randn(self.K, self.D)))
        self.L = LowerTriangularPositive(self.K, self.D)
        self.pi = nn.Parameter(torch.zeros(1, self.K))

    def log_diag_normal(self, x, mu, L, reduction='sum', dim=1):
        m = td.MultivariateNormal(loc=mu,scale_tril=L)
        return m.log_prob(x)

    def forward(self, x):
        x = x.reshape(-1,1,self.D)
        # calculate components
        log_pi = torch.log(F.softmax(self.pi, 1)) # B x K
        log_N = torch.zeros(x.shape[0],self.K,device=x.device) # B x K
        Lt = self.L()
        for i in range(self.K):
```

```
            log_N[:,i:i+1] = self.log_diag_normal(x, torch.sigmoid(self.M[i]),Lt[i])
        NLL_loss = -torch.logsumexp(log_pi + log_N, 1) # B
        return NLL_loss.mean()

    def sample(self, batch_size=6):
        # init an empty tensor
        x_sample = torch.empty(batch_size, self.D)
        # First, sample components
        pi = F.softmax(self.pi, 1) # 1 x K, softmax is used for R^K -> [0,1]^K s.t. sum(pi) = 1
        indices = torch.multinomial(pi, batch_size, replacement=True).squeeze()
        # Then, sample the x given the component
        for n in range(batch_size):
            indx = torch.multinomial(pi, 1, replacement=True).squeeze()
            Lt = self.L()[indx]
            m = td.MultivariateNormal(loc=torch.sigmoid(self.M[indx]),scale_tril=Lt)
            x_sample[n] = m.rsample()
        return x_sample

    def evalP(self, x):
        x = x.reshape(-1,1,self.D)
        # calculate components
        log_pi = torch.log(F.softmax(self.pi, 1)) # B x K
        log_N = torch.zeros(x.shape[0],self.K,device=x.device) # B x K
        Lt = self.L()
        for i in range(self.K):
            log_N[:,i:i+1] = self.log_diag_normal(x, torch.sigmoid(self.M[i]),Lt[i])
        log_prob = torch.logsumexp(log_pi + log_N, 1) # B
        prob = torch.exp(log_prob)
        return prob.squeeze(), log_prob.squeeze()
```

Esta función de entrenamiento es la provista en las notas

```
def training(name, num_epochs, model, optimizer, training_loader):
    nll_min = 0
    # Main loop
    for e in range(num_epochs):
        # TRAINING
        model.train()
        for indx_batch, batch in enumerate(training_loader):
            x,y = batch
            x = x.to('cuda')
            loss = model.forward(x)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
        print(f"{e:03d} {loss.detach().cpu().item():04.4f}")
        if e==0:
            nll_min = loss.item()
        if (loss.item()<nll_min):
            nll_min = loss.item()
            torch.save(model, name + '.model')


D = 784 # input dimension
K = 15 # the number of neurons in scale (s) and translation (t) nets
lr= 1e-4 # learning rate
num_epochs = 500 # max. number of epochs
name = 'mog'


# Eventually, we initialize the full model
model = MoG(D=D, K=K)
model = model.to("cuda")
# Optimizer
optimizer = torch.optim.Adam(model.parameters(), lr=lr)


training(name, num_epochs, model, optimizer, training_loader)
```

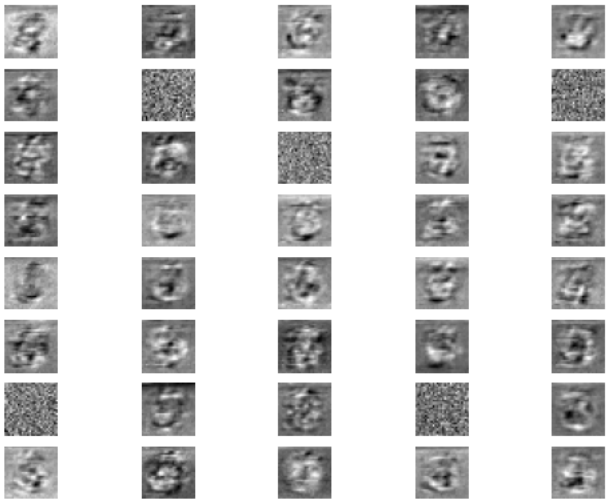
```
 450 -595.4473
 451 -597.6725
 452 -599.8932
 453 -602.1096
 454 -604.3202
 455 -606.5269
 456 -608.7290
 457 -610.9264
 458 -613.1171
 459 -615.3058
 460 -617.4914
 461 -619.6663
 462 -621.8352
 463 -624.0083
 464 -626.1790
 465 -628.3449
 466 -630.5051
 467 -632.6580
 468 -634.8031
 469 -636.9453
 470 -639.0800
 471 -641.2091
 472 -643.3469
 473 -645.4709
 474 -647.5858
 475 -649.6955
 476 -651.7986
 477 -653.9049
 478 -656.0045
 479 -658.1050
 480 -660.2040
 481 -662.2921
 482 -664.3719
 483 -666.4529
 484 -668.5305
 485 -670.6066
 486 -672.6718
 487 -674.7338
 488 -676.7873
 489 -678.8338
 490 -680.8843
 491 -682.9276
 492 -684.9647
 493 -687.0022
 494 -689.0330
 495 -691.0670
 496 -693.0783
 497 -695.0950
 498 -697.1121
 499 -699.1117

with torch.no_grad():
    model_best = torch.load(name + '.model')
    num_x = 8
    num_y = 5
    x = model_best.sample(batch_size=num_x * num_y)
    x = x.detach().numpy()
    fig, ax = plt.subplots(num_x, num_y)
    for i, ax in enumerate(ax.flatten()):
        plottable_image = np.reshape(x[i], (28, 28))
        ax.imshow(plottable_image, cmap='gray')
        ax.axis('off')

    plt.show()

    # Calculate p(x) for the test dataset using the trained model
    px_list = []
    for batch in testing_loader:
        x, _ = batch
        x = x.view(x.size(0), -1).to(model_best.M.device)  # Flatten and move to correct device
        px, _ = model_best.evalP(x)
        px_list.append(px.cpu())
    px_all = torch.cat(px_list)
    print(f"Mean p(x) on test set: {px_all.mean().item():.6f}")
```

```
<ipython-input-9-4cfb2d55f9af>:2: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which
  model_best = torch.load(name + '.model')
```



```
Mean p(x) on test set: inf
```

```
# A very simple Sigmoid Belief Network (SBN) model
# This model uses a logistic regression-like approach
# to model the probability of binary variables.
class FullyVisibleSigmoidBeliefNetwork(torch.nn.Module):
    def __init__(self, input_size):
        super(FullyVisibleSigmoidBeliefNetwork, self).__init__()
        self.input_size = input_size
        self.register_buffer('autoregressive_mask', torch.tril(torch.ones(input_size, input_size), diagonal=-1))
        # Define the logistic regression weights and bias
        self.lr_weights = torch.nn.Parameter(torch.randn(input_size,input_size))
        self.lt_bias = torch.nn.Parameter(torch.randn(input_size))
        # Activation function
        self.sigmoid = nn.Sigmoid()
    # Forward pass of the model
    def forward(self, x):
        # We use a small epsilon to avoid log(0) issues
        # Forward pass evaluates the probability of an x given the alphas
        logits = nn.functional.linear(x,self.lr_weights*self.autoregressive_mask,self.lt_bias)
        return logits
    # Sample from the model
    def sample(self, num_samples):
        samples = torch.zeros(num_samples, self.input_size).to(self.lr_weights.device)
        probs = torch.zeros(num_samples, self.input_size).to(self.lr_weights.device)
        for i in range(self.input_size):
            prob = self.sigmoid(self.forward(samples))
            probs[:, i] = prob[:, i]
            # Sample from the Bernoulli distribution
            samples[:, i] = torch.bernoulli(prob[:, i])
        return samples,probs
    # Calculate p(x) and log p(x) for a given input x
    def evalP(self, x):
        """
        Evaluate the probability p(x) for a given input x.
        x: torch.Tensor of shape [input_size] or [1, input_size]
        Returns: probability p(x) and log-probability log p(x)
        """
        if x.ndim == 1:
            x = x.unsqueeze(0)
        logits = self.forward(x)
        prob_vec = torch.sigmoid(logits)
        # For binary data, p(x) = prod_i p_i^x_i * (1-p_i)^(1-x_i)
        p_x = torch.prod(prob_vec * x + (1 - prob_vec) * (1 - x), dim=1)
        log_p_x = torch.sum(x * torch.log(prob_vec + 1e-8) + (1 - x) * torch.log(1 - prob_vec + 1e-8), dim=1)
        return p_x.squeeze(), log_p_x.squeeze()


def train_models(name, num_epochs, model, optimizer, training_loader):
    min_loss = float('inf')
    for epoch in range(num_epochs):
        model.train()
        for batch_idx, (x, y) in enumerate(training_loader):
```

```
            x = x.view(x.size(0), -1).to("cuda")  # Flatten MNIST images
            logits = model(x)
            # Binary cross-entropy loss
            loss = F.binary_cross_entropy_with_logits(logits, x)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
        print(name + f" Epoch {epoch:03d} Loss: {loss.item():.4f}")
        if loss.item() < min_loss:
            min_loss = loss.item()
            torch.save(model, name + '.model')


# Create a FullyVisibleSigmoidBeliefNetwork for MNIST data
sbn_model = FullyVisibleSigmoidBeliefNetwork(input_size=D).to("cuda")
sbn_optimizer = torch.optim.Adam(sbn_model.parameters(), lr=lr)

# Training loop for SBN model
num_epochs_sbn = 800

train_models('SBN', num_epochs_sbn, sbn_model, sbn_optimizer, training_loader)
```

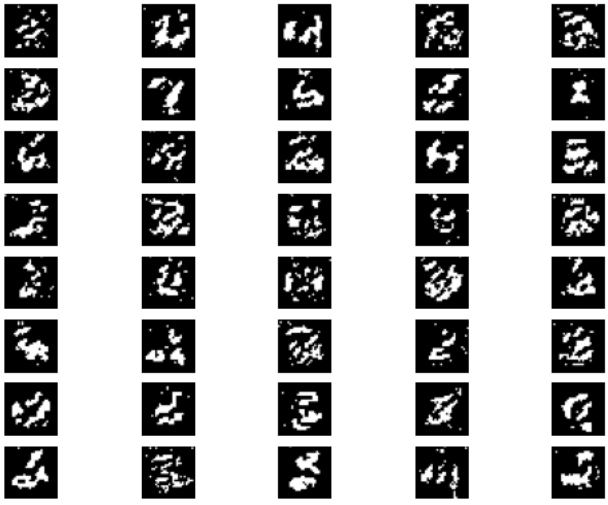
```
SBN Epoch 736 Loss: 0.1125
SBN Epoch 737 Loss: 0.1124
SBN Epoch 738 Loss: 0.1123
SBN Epoch 739 Loss: 0.1123
SBN Epoch 740 Loss: 0.1122
SBN Epoch 741 Loss: 0.1121
SBN Epoch 742 Loss: 0.1120
SBN Epoch 743 Loss: 0.1120
SBN Epoch 744 Loss: 0.1119
SBN Epoch 745 Loss: 0.1118
SBN Epoch 746 Loss: 0.1118
SBN Epoch 747 Loss: 0.1117
SBN Epoch 748 Loss: 0.1116
SBN Epoch 749 Loss: 0.1116
SBN Epoch 750 Loss: 0.1115
SBN Epoch 751 Loss: 0.1114
SBN Epoch 752 Loss: 0.1113
SBN Epoch 753 Loss: 0.1113
SBN Epoch 754 Loss: 0.1112
SBN Epoch 755 Loss: 0.1111
SBN Epoch 756 Loss: 0.1111
SBN Epoch 757 Loss: 0.1110
SBN Epoch 758 Loss: 0.1109
SBN Epoch 759 Loss: 0.1109
SBN Epoch 760 Loss: 0.1108
SBN Epoch 761 Loss: 0.1107
SBN Epoch 762 Loss: 0.1107
SBN Epoch 763 Loss: 0.1106
SBN Epoch 764 Loss: 0.1105
SBN Epoch 765 Loss: 0.1105
SBN Epoch 766 Loss: 0.1104
SBN Epoch 767 Loss: 0.1103
SBN Epoch 768 Loss: 0.1103
SBN Epoch 769 Loss: 0.1102
SBN Epoch 770 Loss: 0.1102
SBN Epoch 771 Loss: 0.1101
SBN Epoch 772 Loss: 0.1100
SBN Epoch 773 Loss: 0.1100
SBN Epoch 774 Loss: 0.1099
SBN Epoch 775 Loss: 0.1098
SBN Epoch 776 Loss: 0.1098
SBN Epoch 777 Loss: 0.1097
SBN Epoch 778 Loss: 0.1097
SBN Epoch 779 Loss: 0.1096
SBN Epoch 780 Loss: 0.1095
SBN Epoch 781 Loss: 0.1095
SBN Epoch 782 Loss: 0.1094
SBN Epoch 783 Loss: 0.1093
SBN Epoch 784 Loss: 0.1093
SBN Epoch 785 Loss: 0.1092
SBN Epoch 786 Loss: 0.1092
SBN Epoch 787 Loss: 0.1091
SBN Epoch 788 Loss: 0.1090
SBN Epoch 789 Loss: 0.1090
SBN Epoch 790 Loss: 0.1089
SBN Epoch 791 Loss: 0.1089
SBN Epoch 792 Loss: 0.1088
SBN Epoch 793 Loss: 0.1088
SBN Epoch 794 Loss: 0.1087
```

```
with torch.no_grad():
    model_best = torch.load('SBN.model')
    num_x = 8
    num_y = 5
    x = model_best.sample(num_samples=num_x * num_y)
    x = x[0].cpu().detach().numpy()  # Move tensor to CPU before converting to numpy
    fig, ax = plt.subplots(num_x, num_y)
    for i, ax in enumerate(ax.flatten()):
        plottable_image = np.reshape(x[i], (28, 28))
        ax.imshow(plottable_image, cmap='gray')
        ax.axis('off')

    plt.show()

    # Calculate p(x) for the test dataset using the trained model
    px_list = []
    for batch in testing_loader:
        x, _ = batch
        x = x.view(x.size(0), -1).to('cuda')  # Flatten and move to correct device
        _, log_px = model_best.evalP(x)
        px_list.append(log_px.cpu())
    px_all = torch.cat(px_list)
    print(f"Mean p(x) on test set: {torch.exp(px_all).mean().item():.6f}")
```

```
<ipython-input-13-cd24c4752d87>:2: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), whic
  model_best = torch.load('SBN.model')
```



```
Mean p(x) on test set: 0.000000
```

```
class NADE(nn.Module):
    """
    NADE with explicit mlp_weights/lr_weights structure and correct masking, including fill().
    """
    def __init__(self, input_size, internal_size=512):
        super().__init__()
        self.input_size = input_size
        self.internal_size = internal_size

        # Degrees for masking
        self.m_input = torch.arange(self.input_size)
        self.m_hidden = torch.from_numpy(
            np.random.randint(0, self.input_size - 1, size=self.internal_size)
        ).sort()[0]

        # Parameters as requested
        self.mlp_weights = nn.Parameter(torch.zeros(self.internal_size, input_size))
        self.mlp_bias = nn.Parameter(torch.zeros(self.internal_size))
        self.lr_weights = nn.Parameter(torch.zeros(input_size, self.internal_size))
        self.lr_bias = nn.Parameter(torch.zeros(input_size))

        nn.init.kaiming_normal_(self.mlp_weights)
        nn.init.kaiming_normal_(self.lr_weights)

        # Masks
        mask1 = (self.m_hidden.unsqueeze(1) >= self.m_input.unsqueeze(0)).float()  # [internal_size, input_size]
        mask2 = (self.m_input.unsqueeze(1) >= self.m_hidden.unsqueeze(0)).float()  # [input_size, internal_size]
```

```
        self.register_buffer("mask1", mask1)
        self.register_buffer("mask2", mask2)

        self.activation = nn.Tanh()
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        if x.shape[1] != self.input_size:
            raise ValueError(f"Expected input of size {self.input_size}, got {x.shape[1]}")
        # Masked input-to-hidden
        h = self.activation(F.linear(x, self.mlp_weights * self.mask1, self.mlp_bias))  # [batch, internal_size]
        # Masked hidden-to-output
        logits = F.linear(h, self.lr_weights * self.mask2, self.lr_bias)  # [batch, input_size]
        return logits

    def sample(self, num_samples):
        device = self.mlp_weights.device
        samples = torch.zeros(num_samples, self.input_size, device=device)
        for i in range(self.input_size):
            logits = self.forward(samples)
            prob = self.sigmoid(logits[:, i])
            samples[:, i] = torch.bernoulli(prob)
        return samples

    def evalP(self, x):
        if x.ndim == 1:
            x = x.unsqueeze(0)
        logits = self.forward(x)
        log_p_x = torch.sum(
            x * F.logsigmoid(logits) + (1 - x) * F.logsigmoid(-logits),
            dim=1
        )
        p_x = torch.exp(log_p_x)
        return p_x.squeeze(), log_p_x.squeeze()


# Train NADE model using Adam optimizer and binary cross-entropy loss on MNIST data
nade_model = NADE(input_size=D, internal_size=512).to("cuda")
nade_optimizer = torch.optim.Adam(nade_model.parameters(), lr=lr)

# Training loop for NADE
num_epochs_nade = 800

train_models('NADE', num_epochs_nade, nade_model, nade_optimizer, training_loader)
```

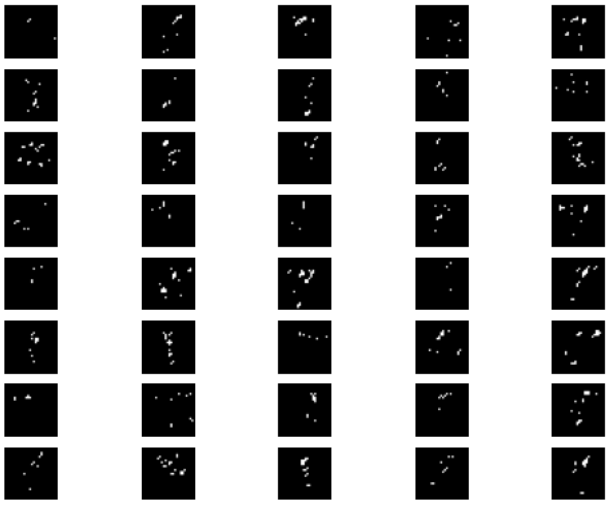
```
NADE Epoch 777 Loss: 0.0832
NADE Epoch 778 Loss: 0.0832
NADE Epoch 779 Loss: 0.0832
NADE Epoch 780 Loss: 0.0832
NADE Epoch 781 Loss: 0.0832
NADE Epoch 782 Loss: 0.0832
NADE Epoch 783 Loss: 0.0832
NADE Epoch 784 Loss: 0.0832
NADE Epoch 785 Loss: 0.0832
NADE Epoch 786 Loss: 0.0832
NADE Epoch 787 Loss: 0.0832
NADE Epoch 788 Loss: 0.0832
NADE Epoch 789 Loss: 0.0831
NADE Epoch 790 Loss: 0.0831
NADE Epoch 791 Loss: 0.0831
NADE Epoch 792 Loss: 0.0831
NADE Epoch 793 Loss: 0.0831
NADE Epoch 794 Loss: 0.0831
NADE Epoch 795 Loss: 0.0831
NADE Epoch 796 Loss: 0.0831
NADE Epoch 797 Loss: 0.0831
NADE Epoch 798 Loss: 0.0831
NADE Epoch 799 Loss: 0.0831
```

```
with torch.no_grad():
    model_best = torch.load('NADE.model')
    num_x = 8
    num_y = 5
    x = model_best.sample(num_samples=num_x * num_y)
    x = x.cpu().detach().numpy()  # Move tensor to CPU before converting to numpy
    fig, ax = plt.subplots(num_x, num_y)
    for i, ax in enumerate(ax.flatten()):
        plottable_image = np.reshape(x[i], (28, 28))
        ax.imshow(plottable_image, cmap='gray')
        ax.axis('off')

    plt.show()

    # Calculate p(x) for the test dataset using the trained model
    px_list = []
    for batch in testing_loader:
        x, _ = batch
        x = x.view(x.size(0), -1).to('cuda')  # Flatten and move to correct device
        _, log_px = model_best.evalP(x)
        px_list.append(log_px.cpu())
    px_all = torch.cat(px_list)
    print(f"Mean p(x) on test set: {torch.exp(px_all).mean().item():.6f}")
```

```
<ipython-input-16-0461d388d225>:2: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), whic
  model_best = torch.load('NADE.model')
```



```
Mean p(x) on test set: 0.000000
```

```
class NADERnn(nn.Module):
    def __init__(self, input_size, internal_size=64):
        super().__init__()
        self.input_size = input_size
        self.internal_size = internal_size
        self.rnn = nn.GRU(1, internal_size, num_layers=1, batch_first=True)
```

```
        # Output weights: one logit per pixel
        self.lr_weights = nn.Parameter(torch.empty(input_size, internal_size))
        self.lr_bias = nn.Parameter(torch.zeros(input_size))

        nn.init.kaiming_normal_(self.lr_weights)
        self.activation = nn.Tanh()

    def forward(self, x):
        """
        Teacher-forced forward pass.
        Autoregressively predicts probabilities for each dimension.
        """
        batch_size = x.size(0)
        h = torch.zeros(1, batch_size, self.internal_size, device=x.device)

        probs = []
        for i in range(self.input_size):
            if i == 0:
                # First pixel: no input yet, just use h0
                h_t = self.activation(h[0])  # [batch, hidden]
            else:
                # Feed previous true pixel into GRU
                out, h = self.rnn(x[:, i-1:i].unsqueeze(-1), h)  # [batch, 1, hidden]
                h_t = self.activation(out[:, -1, :])  # [batch, hidden]

            # Predict prob for pixel i
            logit = h_t @ self.lr_weights[i].unsqueeze(1) + self.lr_bias[i]
            prob = torch.sigmoid(logit)
            probs.append(prob)

        return torch.cat(probs, dim=1)  # [batch, input_size]

    def sample(self, num_samples):
        """
        Sequential ancestral sampling.
        """
        device = self.lr_weights.device
        samples = torch.zeros(num_samples, self.input_size, device=device)
        probs = torch.zeros_like(samples)

        h = torch.zeros(1, num_samples, self.internal_size, device=device)

        for i in range(self.input_size):
            if i == 0:
                h_t = self.activation(h[0])
            else:
                out, h = self.rnn(samples[:, i-1:i].unsqueeze(-1), h)
                h_t = self.activation(out[:, -1, :])

            logit = h_t @ self.lr_weights[i].unsqueeze(1) + self.lr_bias[i]
            prob = torch.sigmoid(logit).squeeze(-1)
            probs[:, i] = prob
            samples[:, i] = torch.bernoulli(prob)

        return samples, probs

    def evalP(self, x):
        """
        Compute p(x) and log p(x).
        """
        if x.ndim == 1:
            x = x.unsqueeze(0)

        prob_vec = self.forward(x)
        p_x = torch.prod(prob_vec * x + (1 - prob_vec) * (1 - x), dim=1)
        log_p_x = torch.sum(
            x * torch.log(prob_vec + 1e-8) +
            (1 - x) * torch.log(1 - prob_vec + 1e-8),
            dim=1
        )
        return p_x.squeeze(), log_p_x.squeeze()



# Train NADERnn model using Adam optimizer and binary cross-entropy loss on MNIST data
nadernn_model = NADERnn(input_size=D, internal_size=16).to("cuda")
```

```
nadernn_optimizer = torch.optim.Adam(nadernn_model.parameters(), lr=lr)

# Training loop for NADERnn
num_epochs_nadern = 100

train_models('NADERnn', num_epochs_nadern, nadernn_model, nadernn_optimizer, training_loader)
```

```
NADERnn Epoch 042 Loss: 0.7160
NADERnn Epoch 043 Loss: 0.7151
NADERnn Epoch 044 Loss: 0.7143
NADERnn Epoch 045 Loss: 0.7135
NADERnn Epoch 046 Loss: 0.7127
NADERnn Epoch 047 Loss: 0.7120
NADERnn Epoch 048 Loss: 0.7113
NADERnn Epoch 049 Loss: 0.7107
NADERnn Epoch 050 Loss: 0.7100
NADERnn Epoch 051 Loss: 0.7094
NADERnn Epoch 052 Loss: 0.7089
NADERnn Epoch 053 Loss: 0.7083
NADERnn Epoch 054 Loss: 0.7078
NADERnn Epoch 055 Loss: 0.7073
NADERnn Epoch 056 Loss: 0.7068
NADERnn Epoch 057 Loss: 0.7063
NADERnn Epoch 058 Loss: 0.7059
NADERnn Epoch 059 Loss: 0.7055
NADERnn Epoch 060 Loss: 0.7051
NADERnn Epoch 061 Loss: 0.7047
NADERnn Epoch 062 Loss: 0.7043
NADERnn Epoch 063 Loss: 0.7040
NADERnn Epoch 064 Loss: 0.7036
NADERnn Epoch 065 Loss: 0.7033
NADERnn Epoch 066 Loss: 0.7030
NADERnn Epoch 067 Loss: 0.7027
NADERnn Epoch 068 Loss: 0.7024
NADERnn Epoch 069 Loss: 0.7021
NADERnn Epoch 070 Loss: 0.7018
NADERnn Epoch 071 Loss: 0.7015
NADERnn Epoch 072 Loss: 0.7013
NADERnn Epoch 073 Loss: 0.7010
NADERnn Epoch 074 Loss: 0.7008
NADERnn Epoch 075 Loss: 0.7006
NADERnn Epoch 076 Loss: 0.7004
NADERnn Epoch 077 Loss: 0.7001
NADERnn Epoch 078 Loss: 0.6999
NADERnn Epoch 079 Loss: 0.6997
NADERnn Epoch 080 Loss: 0.6995
NADERnn Epoch 081 Loss: 0.6994
NADERnn Epoch 082 Loss: 0.6992
NADERnn Epoch 083 Loss: 0.6990
NADERnn Epoch 084 Loss: 0.6988
NADERnn Epoch 085 Loss: 0.6987
NADERnn Epoch 086 Loss: 0.6985
NADERnn Epoch 087 Loss: 0.6984
NADERnn Epoch 088 Loss: 0.6982
NADERnn Epoch 089 Loss: 0.6981
NADERnn Epoch 090 Loss: 0.6979
NADERnn Epoch 091 Loss: 0.6978
NADERnn Epoch 092 Loss: 0.6977
NADERnn Epoch 093 Loss: 0.6976
NADERnn Epoch 094 Loss: 0.6974
NADERnn Epoch 095 Loss: 0.6973
NADERnn Epoch 096 Loss: 0.6972
NADERnn Epoch 097 Loss: 0.6971
NADERnn Epoch 098 Loss: 0.6970
NADERnn Epoch 099 Loss: 0.6969
```

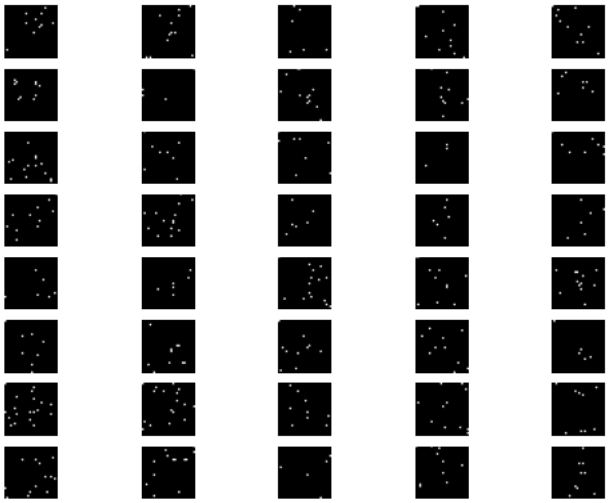
```
with torch.no_grad():
    model_best = torch.load('NADERnn.model')
    num_x = 8
    num_y = 5
    x = model_best.sample(num_samples=num_x * num_y)
    x = x[0].cpu().detach().numpy()  # Move tensor to CPU before converting to numpy
    fig, ax = plt.subplots(num_x, num_y)
    for i, ax in enumerate(ax.flatten()):
        plottable_image = np.reshape(x[i], (28, 28))
        ax.imshow(plottable_image, cmap='gray')
        ax.axis('off')

    plt.show()

    # Calculate p(x) for the test dataset using the trained model
    px_list = []
```

```
    for batch in testing_loader:
        x, _ = batch
        x = x.view(x.size(0), -1).to('cuda')  # Flatten and move to correct device
        _, log_px = model_best.evalP(x)
        px_list.append(log_px.cpu())
    px_all = torch.cat(px_list)
    print(f"Mean p(x) on test set: {torch.exp(px_all).mean().item():.6f}")
```

```
<ipython-input-19-387dd220cee8>:2: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), whic
  model_best = torch.load('NADERnn.model')
```



```
Mean p(x) on test set: 0.000000
```

```
class CausalConv1d(nn.Module):

    """
    A causal 1D convolution.
    Implementation from Tomczak.
    """
    def __init__(self, in_channels, out_channels, kernel_size, dilation,exclude_current=False, **kwargs):
        super(CausalConv1d, self).__init__()
        # attributes:
        self.kernel_size = kernel_size
        self.dilation = dilation
        self.exclude_current = exclude_current
        self.padding = (kernel_size - 1) * dilation + (1 if exclude_current else 0) * dilation
        # module:
        self.conv1d = torch.nn.Conv1d(in_channels, out_channels,
                                      kernel_size, stride=1,
                                      padding=0,
                                      dilation=dilation,
                                      **kwargs)

    def forward(self, x):
        # Here is the trick!
        x = torch.nn.functional.pad(x, (self.padding, 0))
        conv1d_out = self.conv1d(x)
        if self.exclude_current:
            return conv1d_out[:, :, : -1]
        else:
            return conv1d_out


class CausalConvARM(nn.Module):
    def __init__(self, input_size=2):
        super(CausalConvARM, self).__init__()
        self.kernel_size = 11
        self.n_channels = 32
        self.sigmoid = nn.Sigmoid()
        self.net = nn.Sequential(
            CausalConv1d(in_channels=1, out_channels=self.n_channels,dilation=1, kernel_size=self.kernel_size, exclude_current=True, bias=Tr
            nn.LeakyReLU(),
            CausalConv1d(in_channels=self.n_channels, out_channels=self.n_channels, dilation=2, kernel_size=self.kernel_size, exclude_curren
            nn.LeakyReLU(),
            CausalConv1d(in_channels=self.n_channels, out_channels=self.n_channels, dilation=2, kernel_size=self.kernel_size, exclude_curren
            nn.LeakyReLU(),
```

```
            CausalConv1d(in_channels=self.n_channels, out_channels=self.n_channels, dilation=1, kernel_size=self.kernel_size, exclude_curren
        )
        # Define the logistic regression weights and bias
        self.lr_weights = torch.nn.Parameter(torch.zeros(input_size,self.n_channels))
        self.lr_bias = torch.nn.Parameter(torch.zeros(input_size))
        self.input_size = input_size

    def forward(self, x):
        h = self.net(x.unsqueeze(1))
        h = h.permute(0, 2, 1)
        sp= torch.sum(h*self.lr_weights,dim=2)+self.lr_bias
        return sp.squeeze(-1)

    def sample(self, num_samples):
        device = next(self.parameters()).device
        samples = torch.zeros(num_samples, self.input_size).to(device)
        probs = torch.zeros(num_samples, self.input_size).to(device)
        for i in range(self.input_size):
            logits = self.forward(samples)
            prob = self.sigmoid(logits)
            probs[:, i] = prob[:,i]
            # Sample from the Bernoulli distribution
            samples[:, i] = torch.bernoulli(probs[:, i])
        return samples, probs
    # Calculate p(x) and log p(x) for a given input x
    def evalP(self, x):
        """
        Evaluate the probability p(x) for a given input x.
        x: torch.Tensor of shape [input_size] or [1, input_size]
        Returns: probability p(x) and log-probability log p(x)
        """
        if x.ndim == 1:
            x = x.unsqueeze(0)
        logits = self.forward(x)
        prob_vec = torch.sigmoid(logits)
        p_x = torch.prod(prob_vec * x + (1 - prob_vec) * (1 - x), dim=1)
        log_p_x = torch.sum(x * torch.log(prob_vec + 1e-8) + (1 - x) * torch.log(1 - prob_vec + 1e-8), dim=1)
        return p_x.squeeze(), log_p_x.squeeze()


# Train CausalConvARM model using Adam optimizer and binary cross-entropy loss on MNIST data
causalconvarm_model = CausalConvARM(input_size=D).to("cuda")
causalconvarm_optimizer = torch.optim.Adam(causalconvarm_model.parameters(), lr=lr)

# Training loop for CausalConvARM
num_epochs_causalconvarm = 800
train_models("CausalConvARM", num_epochs_causalconvarm, causalconvarm_model, causalconvarm_optimizer, training_loader)
```

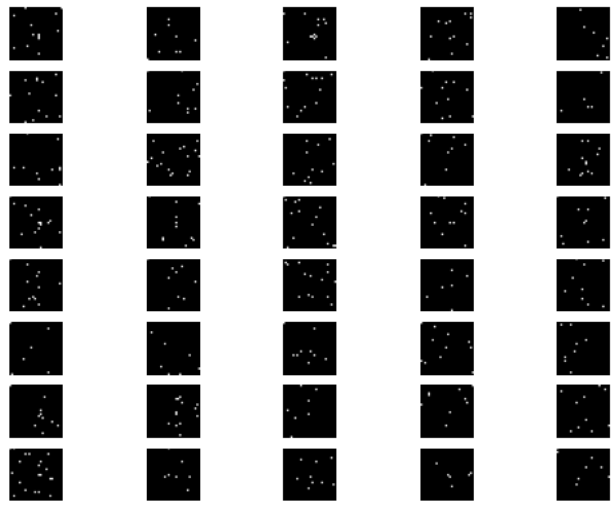
CausalConvARM Epoch 772 Loss: 0.0800

```
CausalConvARM Epoch 773 Loss: 0.0800
CausalConvARM Epoch 774 Loss: 0.0800
CausalConvARM Epoch 775 Loss: 0.0800
CausalConvARM Epoch 776 Loss: 0.0800
CausalConvARM Epoch 777 Loss: 0.0800
CausalConvARM Epoch 778 Loss: 0.0800
CausalConvARM Epoch 779 Loss: 0.0800
CausalConvARM Epoch 780 Loss: 0.0800
CausalConvARM Epoch 781 Loss: 0.0800
CausalConvARM Epoch 782 Loss: 0.0800
CausalConvARM Epoch 783 Loss: 0.0800
CausalConvARM Epoch 784 Loss: 0.0800
CausalConvARM Epoch 785 Loss: 0.0800
CausalConvARM Epoch 786 Loss: 0.0800
CausalConvARM Epoch 787 Loss: 0.0800
CausalConvARM Epoch 788 Loss: 0.0800
CausalConvARM Epoch 789 Loss: 0.0800
CausalConvARM Epoch 790 Loss: 0.0800
CausalConvARM Epoch 791 Loss: 0.0800
CausalConvARM Epoch 792 Loss: 0.0800
CausalConvARM Epoch 793 Loss: 0.0800
CausalConvARM Epoch 794 Loss: 0.0800
CausalConvARM Epoch 795 Loss: 0.0800
CausalConvARM Epoch 796 Loss: 0.0800
CausalConvARM Epoch 797 Loss: 0.0800
CausalConvARM Epoch 798 Loss: 0.0800
CausalConvARM Epoch 799 Loss: 0.0800
```

```
with torch.no_grad():
    num_x = 8
    num_y = 5
    x = model_best.sample(num_samples=num_x * num_y)
    x = x[0].cpu().detach().numpy()  # Move tensor to CPU before converting to numpy
    fig, ax = plt.subplots(num_x, num_y)
    for i, ax in enumerate(ax.flatten()):
        plottable_image = np.reshape(x[i], (28, 28))
        ax.imshow(plottable_image, cmap='gray')
        ax.axis('off')

    plt.show()

    # Calculate p(x) for the test dataset using the trained model
    px_list = []
    for batch in testing_loader:
        x, _ = batch
        x = x.view(x.size(0), -1).to('cuda')  # Flatten and move to correct device
        _, log_px = model_best.evalP(x)
        px_list.append(log_px.cpu())
    px_all = torch.cat(px_list)
    print(f"Mean p(x) on test set: {torch.exp(px_all).mean().item():.6f}")
```



```
Mean p(x) on test set: 0.000000
```

Ninguna de las redes funcionó de acuerdo a lo esperado. Es posible que fueran sobre-entrenadas o que requirieran mucho más entrenamiento, pero $p(x)$ en promedio es cero para todas, no he encontrado ningún error después de varios días buscando, por lo que si pudiera decirme qué falta, se lo agradecería. El modelo de NADE lo reescribí pues tardaba demasiado el entrenamiento, pero conservé la estructura original.

Problema 2

Consideramos una factorización bivarioada para el modelo NADE. Para hacerlo, proponemos que la salida sea la probabilidad de cada uno de los siguientes casos:

$$(X_1, X_2) = (0, 0), \qquad (X_1, X_2) = (0, 1)$$
$$(X_1, X_2) = (1, 0), \qquad (X_1, X_2) = (1, 1)$$

La función de pérdida sigue siendo la entropía cruzada.

```
class BivariateNADE(nn.Module):
    """
    Bivariate NADE with explicit mlp_weights/lr_weights structure and correct masking.
    Output: 4 logits per pixel pair.
    """
    def __init__(self, input_size, internal_size=512):
        super().__init__()

        # Ensure even input size (pairs of pixels)
        self.input_size = input_size if input_size % 2 == 0 else input_size + 1
        self.internal_size = internal_size
        self.n_pairs = self.input_size // 2

        # Degrees
        self.m_input = torch.arange(self.input_size)  # 0..D-1
        self.m_hidden = torch.from_numpy(
            np.random.randint(0, self.input_size - 1, size=self.internal_size)
        ).sort()[0]

        # Parameters (same structure as NADE)
        self.mlp_weights = nn.Parameter(torch.zeros(self.internal_size, self.input_size))
        self.mlp_bias = nn.Parameter(torch.zeros(self.internal_size))
        self.lr_weights = nn.Parameter(torch.zeros(4 * self.n_pairs, self.internal_size))
        self.lr_bias = nn.Parameter(torch.zeros(4 * self.n_pairs))

        nn.init.kaiming_normal_(self.mlp_weights)
        nn.init.kaiming_normal_(self.lr_weights)

        # Mask 1: input -> hidden
        mask1 = (self.m_hidden.unsqueeze(1) >= self.m_input.unsqueeze(0)).float()  # [internal_size, input_size]

        # Mask 2: hidden -> output
        m_output_pairs = torch.arange(self.n_pairs) * 2 + 1  # degree for each pair
        mask2 = (m_output_pairs.unsqueeze(1) >= self.m_hidden.unsqueeze(0)).float()  # [n_pairs, internal_size]
        mask2 = mask2.repeat_interleave(4, dim=0)  # [4*n_pairs, internal_size]

        self.register_buffer("mask1", mask1)
        self.register_buffer("mask2", mask2)

        self.activation = nn.Tanh()

    def forward(self, x):
        """
        Forward pass: returns logits for each pair.
        x: [batch, input_size]
        output: [batch, n_pairs, 4]
        """
        if x.shape[1] != self.input_size:
            pad = self.input_size - x.shape[1]
            x = F.pad(x, (0, pad), "constant", 0)

        h = self.activation(F.linear(x, self.mlp_weights * self.mask1, self.mlp_bias))  # [batch, internal_size]
        logits = F.linear(h, self.lr_weights * self.mask2, self.lr_bias)  # [batch, 4*n_pairs]
        logits = logits.view(-1, self.n_pairs, 4)
        return logits

    def sample(self, num_samples):
        """
        Sequentially sample pairs according to autoregressive ordering.
        """
        device = self.mlp_weights.device
        samples = torch.zeros(num_samples, self.input_size, device=device)

        for i in range(self.n_pairs):
```

```
            logits = self.forward(samples)  # [num_samples, n_pairs, 4]
            logits_pair = logits[:, i, :]   # [num_samples, 4]
            probs = F.softmax(logits_pair, dim=-1)
            idx = torch.multinomial(probs, 1).squeeze(1)  # [num_samples]
            # Decode idx to two bits
            samples[:, 2*i]   = (idx >> 1) & 1
            samples[:, 2*i+1] = idx & 1

        return samples[:, :self.input_size]

    def evalP(self, x):
        if x.ndim == 1:
            x = x.unsqueeze(0)
        if x.shape[1] != self.input_size:
            pad = self.input_size - x.shape[1]
            x = F.pad(x, (0, pad), "constant", 0)
        logits = self.forward(x)  # [batch, n_pairs, 4]
        probs = F.softmax(logits, dim=-1)  # [batch, n_pairs, 4]

        # Compute indices for all pairs in batch
        pair_bits = x.view(x.shape[0], self.n_pairs, 2).long()
        idx = (pair_bits[:,:,0] << 1) + pair_bits[:,:,1]  # [batch, n_pairs]

        # Gather probabilities for each pair
        px = probs.gather(2, idx.unsqueeze(-1)).squeeze(-1)  # [batch, n_pairs]
        log_px = torch.log(px + 1e-8)  # [batch, n_pairs]

        # Product and sum over pairs
        p_x = torch.prod(px, dim=1)
        log_p_x = torch.sum(log_px, dim=1)
        return p_x.squeeze(), log_p_x.squeeze()


# Train bivariateNADE model using Adam optimizer and cross-entropy loss on MNIST data
bivnade_model = BivariateNADE(input_size=D, internal_size=512).to("cuda")
bivnade_optimizer = torch.optim.Adam(bivnade_model.parameters(), lr=lr)

num_epochs_bivnade = 500

bv_min_loss = float('inf')
for epoch in range(num_epochs_bivnade):
    bivnade_model.train()
    for batch_idx, (x, y) in enumerate(training_loader):
        x = x.view(x.size(0), -1).to("cuda")  # Flatten MNIST images
        # Pad if needed
        if x.shape[1] % 2 != 0:
            x = F.pad(x, (0, 1), 'constant', 0)
        logits = bivnade_model(x)  # [batch, n_pairs, 4]
        # Prepare targets: for each pair, get index 0-3
        targets = []
        for i in range(bivnade_model.n_pairs):
            pair = x[:, 2*i:2*i+2]
            idx = (pair[:, 0].long() << 1) + pair[:, 1].long()
            targets.append(idx)
        targets = torch.stack(targets, dim=1)  # [batch, n_pairs]
        # Reshape logits and targets for cross-entropy
        logits_flat = logits.view(-1, 4)
        targets_flat = targets.view(-1)
        bv_loss = F.cross_entropy(logits_flat, targets_flat)
        bivnade_optimizer.zero_grad()
        bv_loss.backward()
        bivnade_optimizer.step()
        if bv_loss.item() < bv_min_loss:
            bv_min_loss = bv_loss.item()
            torch.save(bivnade_model, 'bivnade.model')
    print(f"Bivariate NADE Epoch {epoch:03d} Loss: {bv_loss.item():.4f}")
```

```
Bivariate NADE Epoch 299 Loss: 0.0468
```

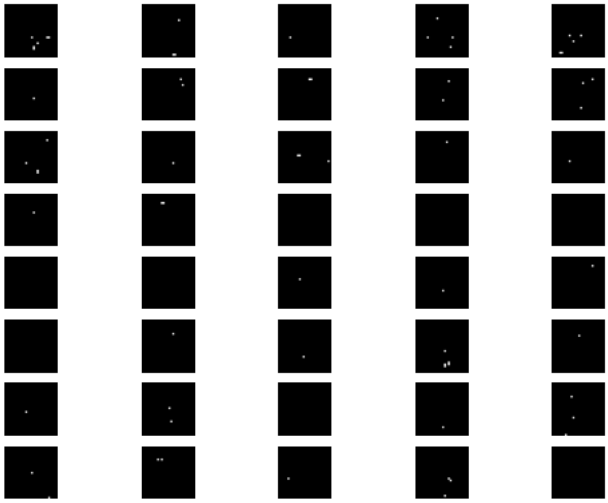
```
Bivariate NADE Epoch 309 Loss: 0.0468
Bivariate NADE Epoch 310 Loss: 0.0468
Bivariate NADE Epoch 311 Loss: 0.0468
Bivariate NADE Epoch 312 Loss: 0.0467
Bivariate NADE Epoch 313 Loss: 0.0467
Bivariate NADE Epoch 314 Loss: 0.0467
Bivariate NADE Epoch 315 Loss: 0.0467
Bivariate NADE Epoch 316 Loss: 0.0466
Bivariate NADE Epoch 317 Loss: 0.0466
Bivariate NADE Epoch 318 Loss: 0.0466
Bivariate NADE Epoch 319 Loss: 0.0465
Bivariate NADE Epoch 320 Loss: 0.0465
Bivariate NADE Epoch 321 Loss: 0.0465
Bivariate NADE Epoch 322 Loss: 0.0465
Bivariate NADE Epoch 323 Loss: 0.0464
Bivariate NADE Epoch 324 Loss: 0.0464
Bivariate NADE Epoch 325 Loss: 0.0464
Bivariate NADE Epoch 326 Loss: 0.0463
Bivariate NADE Epoch 327 Loss: 0.0463
Bivariate NADE Epoch 328 Loss: 0.0463
Bivariate NADE Epoch 329 Loss: 0.0463
Bivariate NADE Epoch 330 Loss: 0.0462
Bivariate NADE Epoch 331 Loss: 0.0462
Bivariate NADE Epoch 332 Loss: 0.0462
Bivariate NADE Epoch 333 Loss: 0.0462
Bivariate NADE Epoch 334 Loss: 0.0461
Bivariate NADE Epoch 335 Loss: 0.0461
Bivariate NADE Epoch 336 Loss: 0.0461
Bivariate NADE Epoch 337 Loss: 0.0460
Bivariate NADE Epoch 338 Loss: 0.0460
Bivariate NADE Epoch 339 Loss: 0.0460
Bivariate NADE Epoch 340 Loss: 0.0460
Bivariate NADE Epoch 341 Loss: 0.0459
Bivariate NADE Epoch 342 Loss: 0.0459
Bivariate NADE Epoch 343 Loss: 0.0459
Bivariate NADE Epoch 344 Loss: 0.0459
Bivariate NADE Epoch 345 Loss: 0.0458
Bivariate NADE Epoch 346 Loss: 0.0458
Bivariate NADE Epoch 347 Loss: 0.0458
Bivariate NADE Epoch 348 Loss: 0.0458
Bivariate NADE Epoch 349 Loss: 0.0457
Bivariate NADE Epoch 350 Loss: 0.0457
Bivariate NADE Epoch 351 Loss: 0.0457
Bivariate NADE Epoch 352 Loss: 0.0457
Bivariate NADE Epoch 353 Loss: 0.0456
Bivariate NADE Epoch 354 Loss: 0.0456
Bivariate NADE Epoch 355 Loss: 0.0456
```

```
with torch.no_grad():
    model_best = torch.load('bivnade.model')
    num_x = 8
    num_y = 5
    x = model_best.sample(num_samples=num_x * num_y)
    x = x.cpu().detach().numpy()
    fig, ax = plt.subplots(num_x, num_y)
    for i, ax in enumerate(ax.flatten()):
        plottable_image = np.reshape(x[i], (28, 28))
        ax.imshow(plottable_image, cmap='gray')
        ax.axis('off')

    plt.show()

    # Calculate p(x) for the test dataset using the trained model
    px_list = []
    for batch in testing_loader:
        x, _ = batch
        x = x.view(x.size(0), -1).to('cuda')  # Flatten and move to correct device
        px, _ = model_best.evalP(x)
        px_list.append(px.cpu())
    px_all = torch.cat(px_list)
    print(f"Mean p(x) on test set: {px_all.mean().item():.6f}")
```

```
<ipython-input-26-55413bc5a8d4>:2: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), whic
  model_best = torch.load('bivnade.model')
```



```
Mean p(x) on test set: 0.005787
```

En este caso, $p(x)$ no es cero en promedio, pero si es bastante baja. No hay una clara diferencia entre los dos modelos, si acaso el NADE original es cualitativamente mejor.

Problema 3

Añadimos la función fill a la clase de NADE, donde utilizamos una máscara para elegir si completamos un dígito o utilizamos el provisto. Por la implementación, es posible considerar imágenes con otra estructura faltante.

```
class NADE(nn.Module):
    """
    NADE with explicit mlp_weights/lr_weights structure and correct masking, including fill().
    """
    def __init__(self, input_size, internal_size=512):
        super().__init__()
        self.input_size = input_size
        self.internal_size = internal_size

        # Degrees for masking
        self.m_input = torch.arange(self.input_size)
        self.m_hidden = torch.from_numpy(
            np.random.randint(0, self.input_size - 1, size=self.internal_size)
        ).sort()[0]

        # Parameters as requested
        self.mlp_weights = nn.Parameter(torch.zeros(self.internal_size, input_size))
        self.mlp_bias = nn.Parameter(torch.zeros(self.internal_size))
        self.lr_weights = nn.Parameter(torch.zeros(input_size, self.internal_size))
        self.lr_bias = nn.Parameter(torch.zeros(input_size))

        nn.init.kaiming_normal_(self.mlp_weights)
        nn.init.kaiming_normal_(self.lr_weights)

        # Masks
        mask1 = (self.m_hidden.unsqueeze(1) >= self.m_input.unsqueeze(0)).float()  # [internal_size, input_size]
        mask2 = (self.m_input.unsqueeze(1) >= self.m_hidden.unsqueeze(0)).float()  # [input_size, internal_size]
        self.register_buffer("mask1", mask1)
        self.register_buffer("mask2", mask2)

        self.activation = nn.Tanh()
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        if x.shape[1] != self.input_size:
            raise ValueError(f"Expected input of size {self.input_size}, got {x.shape[1]}")
        # Masked input-to-hidden
        h = self.activation(F.linear(x, self.mlp_weights * self.mask1, self.mlp_bias))  # [batch, internal_size]
        # Masked hidden-to-output
```

```
        logits = F.linear(h, self.lr_weights * self.mask2, self.lr_bias)  # [batch, input_size]
        return logits

    def sample(self, num_samples):
        device = self.mlp_weights.device
        samples = torch.zeros(num_samples, self.input_size, device=device)
        for i in range(self.input_size):
            logits = self.forward(samples)
            prob = self.sigmoid(logits[:, i])
            samples[:, i] = torch.bernoulli(prob)
        return samples

    def evalP(self, x):
        if x.ndim == 1:
            x = x.unsqueeze(0)
        logits = self.forward(x)
        log_p_x = torch.sum(
            x * F.logsigmoid(logits) + (1 - x) * F.logsigmoid(-logits),
            dim=1
        )
        p_x = torch.exp(log_p_x)
        return p_x.squeeze(), log_p_x.squeeze()

    def fill(self, x_partial, mask=None):
        """
        Fill missing pixels in an image using the NADE model.
        Args:
            x_partial: [input_size] or [batch, input_size], known pixels set, unknowns = 0 or any value.
            mask: same shape, 1=known, 0=unknown. If None, treat zeros as unknowns.
        Returns:
            x_filled: completed image
        """
        if x_partial.ndim == 1:
            x_partial = x_partial.unsqueeze(0)
        batch_size, input_size = x_partial.shape
        x_filled = x_partial.clone()
        if mask is None:
            mask = (x_partial != 0).float()
        for i in range(input_size):
            missing = (mask[:, i] == 0)
            if missing.any():
                logits = self.forward(x_filled)
                prob = self.sigmoid(logits)[:, i]
                x_filled[missing, i] = torch.bernoulli(prob[missing])
                mask[missing, i] = 1
        return x_filled.squeeze(0) if batch_size == 1 else x_filled
```

Cargamos el mejor modelo en vez de entrenar nuevamente.

```
# Train NADE model using Adam optimizer and binary cross-entropy loss on MNIST data
nade_model = NADE(input_size=D, internal_size=512).to("cuda")

# Load trained NADE model weights
nade_model.load_state_dict(torch.load('NADE.model').state_dict())
nade_model.eval()
```

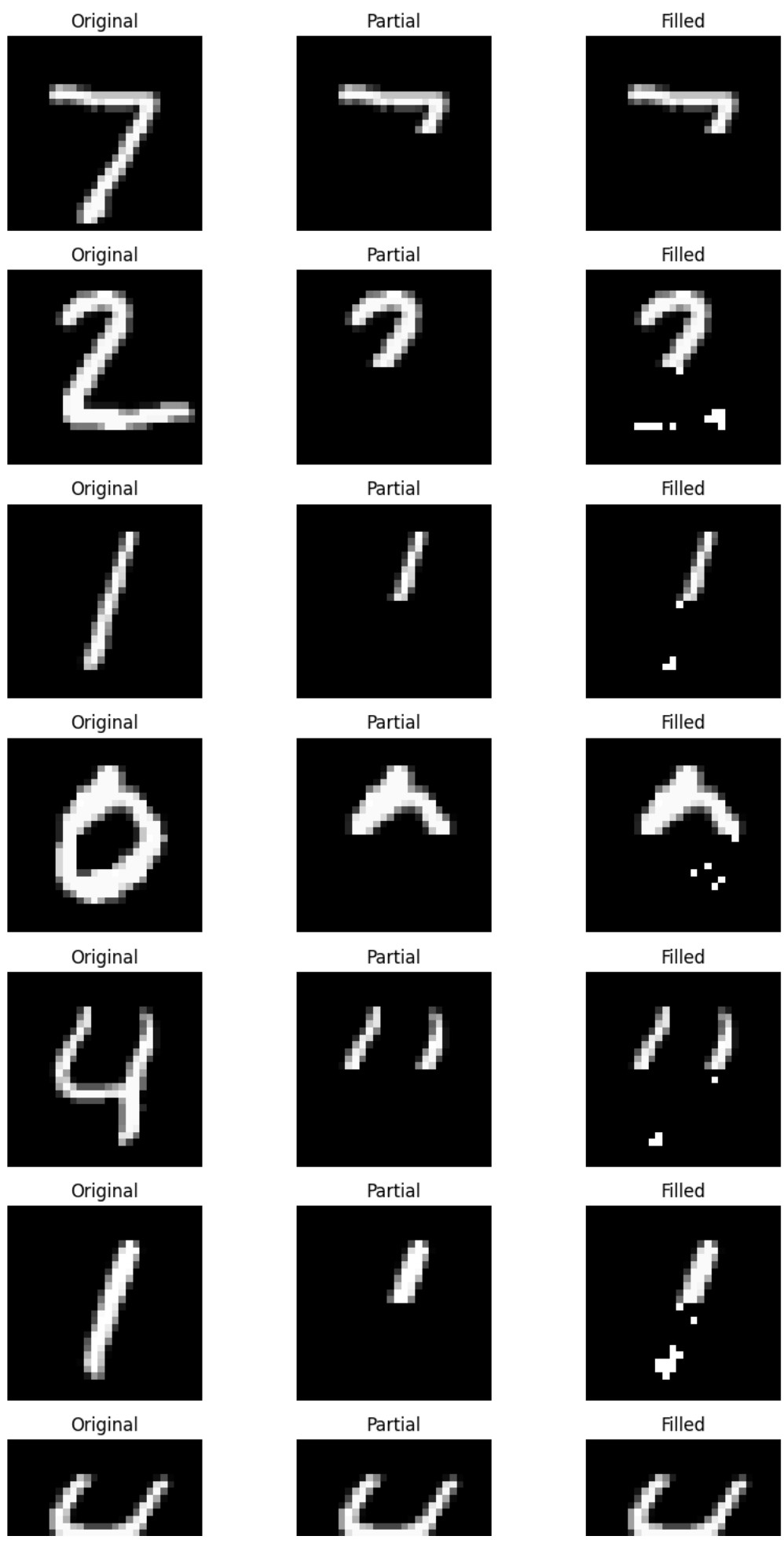
```
<ipython-input-28-2c80e9794ed7>:5: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), whic
  nade_model.load_state_dict(torch.load('NADE.model').state_dict())
NADE(
  (activation): Tanh()
  (sigmoid): Sigmoid()
)
```

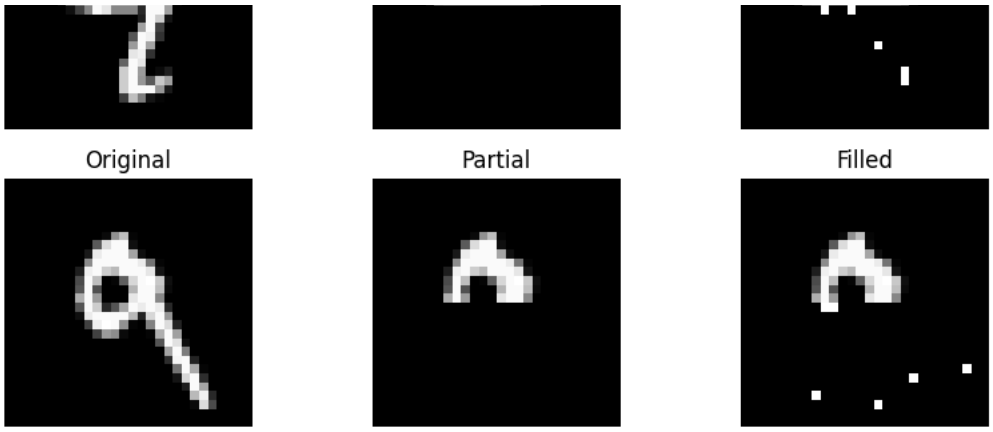
```
with torch.no_grad():
    # Get a batch from the testing dataset
    for batch in testing_loader:
        x, y = batch
        x = x.view(x.size(0), 28, 28)  # Reshape to [batch, 28, 28]
        # Only use the first 8 images for visualization
        x = x[:8]
        # Create partial images: keep first 14 rows, zero out the rest
        x_partial = x.clone()
        x_partial[:, 14:, :] = 0

        # Define a mask where known pixels are marked as 1 and unknown as 0
```

```
mask = torch.ones_like(x)
mask[:, 14:, :] = 0

# Flatten for NADE input
x_partial_flat = x_partial.view(x_partial.size(0), -1).to('cuda')
mask_flat = mask.view(mask.size(0), -1).to('cuda')
# Fill missing pixels
x_filled_flat = nade_model.fill(x_partial_flat, mask = mask_flat)
x_filled = x_filled_flat.cpu().numpy().reshape(-1, 28, 28)
# Visualize original, partial, and filled images
fig, axes = plt.subplots(8, 3, figsize=(9, 18))
for i in range(8):
    axes[i, 0].imshow(x[i].cpu().numpy(), cmap='gray')
    axes[i, 0].set_title('Original')
    axes[i, 0].axis('off')
    axes[i, 1].imshow(x_partial[i].cpu().numpy(), cmap='gray')
    axes[i, 1].set_title('Partial')
    axes[i, 1].axis('off')
    axes[i, 2].imshow(x_filled[i], cmap='gray')
    axes[i, 2].set_title('Filled')
    axes[i, 2].axis('off')
plt.tight_layout()
plt.show()
break  # Only process one batch
```


Original
Partial
Filled
Original
Partial
Filled
Original
Partial
Filled
Original
Partial
Filled
Original
Partial
Filled
Original
Partial
Filled
Original
Partial
Filled



Para el problema de completar, podemos identificar algunos números, pero en general seguimos teniendo un mal desempeño.

Problema 4

Añadimos la posibilidad de permutar los datos, tanto al entrenamiento, como a la función de fill previamente creada. Proponemos las siguientes permutaciones:

- Recorrer verticalmente desde el inicio.
- Recorrer horizontalmente desde el último pixel
- Recorrer horizontalmente desde la fila central, hacia abajo y hacia arriba alternadamente
- Recorrer verticalmente desde la columna central, hacia la derecha e izquierda alternadamente
- Una espiral desde los extremos hacia el centro

```
class NADE(nn.Module):
    """
    NADE with explicit mlp_weights/lr_weights structure and correct masking, including fill().
    """
    def __init__(self, input_size, internal_size=512, permutation=None):
        super().__init__()
        self.input_size = input_size
        self.internal_size = internal_size

        # Degrees for masking
        self.m_input = torch.arange(self.input_size)
        self.m_hidden = torch.from_numpy(
            np.random.randint(0, self.input_size - 1, size=self.internal_size)
        ).sort()[0]

        # Parameters as requested
        self.mlp_weights = nn.Parameter(torch.zeros(self.internal_size, input_size))
        self.mlp_bias = nn.Parameter(torch.zeros(self.internal_size))
        self.lr_weights = nn.Parameter(torch.zeros(input_size, self.internal_size))
        self.lr_bias = nn.Parameter(torch.zeros(input_size))

        nn.init.kaiming_normal_(self.mlp_weights)
        nn.init.kaiming_normal_(self.lr_weights)

        # Masks
        mask1 = (self.m_hidden.unsqueeze(1) >= self.m_input.unsqueeze(0)).float()  # [internal_size, input_size]
        mask2 = (self.m_input.unsqueeze(1) >= self.m_hidden.unsqueeze(0)).float()  # [input_size, internal_size]
        self.register_buffer("mask1", mask1)
        self.register_buffer("mask2", mask2)

        self.activation = nn.Tanh()
        self.sigmoid = nn.Sigmoid()

        # Permutation: if None, identity
        if permutation is None:
            self.register_buffer("permutation", torch.arange(input_size))
        else:
            self.register_buffer("permutation", torch.tensor(permutation, dtype=torch.long))

    def forward(self, x):
        if x.shape[1] != self.input_size:
```

```
            raise ValueError(f"Expected input of size {self.input_size}, got {x.shape[1]}")

        x_perm = x[:, self.permutation] if x.ndim == 2 else x[self.permutation]

        # Masked input-to-hidden
        h = self.activation(F.linear(x_perm, self.mlp_weights * self.mask1, self.mlp_bias))  # [batch, internal_size]

        # Masked hidden-to-output
        logits = F.linear(h, self.lr_weights * self.mask2, self.lr_bias)  # [batch, input_size]
        inv_perm = torch.argsort(self.permutation)
        logits = logits[:, inv_perm] if logits.ndim == 2 else logits[inv_perm]
        return logits


    def sample(self, num_samples):
        device = self.mlp_weights.device
        samples = torch.zeros(num_samples, self.input_size, device=device)
        for idx in self.permutation:
            logits = self.forward(samples)
            prob = self.sigmoid(logits[:, idx])
            samples[:, idx] = torch.bernoulli(prob)
        return samples


    def evalP(self, x):
        if x.ndim == 1:
            x = x.unsqueeze(0)
        logits = self.forward(x)
        log_p_x = torch.sum(
            x * F.logsigmoid(logits) + (1 - x) * F.logsigmoid(-logits),
            dim=1
        )
        p_x = torch.exp(log_p_x)
        return p_x.squeeze(), log_p_x.squeeze()

    def fill(self, x_partial, mask=None):
        if x_partial.ndim == 1:
            x_partial = x_partial.unsqueeze(0)
        batch_size, input_size = x_partial.shape
        x_filled = x_partial.clone()
        if mask is None:
            mask = (x_partial != 0).float()
        for idx in self.permutation:
            missing = (mask[:, idx] == 0)
            if missing.any():
                logits = self.forward(x_filled)
                prob = self.sigmoid(logits)[:, idx]
                x_filled[missing, idx] = torch.bernoulli(prob[missing])
                mask[missing, idx] = 1
        return x_filled.squeeze(0) if batch_size == 1 else x_filled
```

Aquí utilicé 200 épocas por cada permutación, y al final comparamos.

```
# p1: permutation for vertical (column-major) order for MNIST 28x28 images
p1 = [i + 28*j for i in range(28) for j in range(28)]
p1_nade_model = NADE(input_size=D, internal_size=512, permutation=p1).to("cuda")

p1_nade_optimizer = torch.optim.Adam(p1_nade_model.parameters(), lr=lr)

# Training loop for permutated NADEs
num_epochs_nade_p = 200

train_models('P1_NADE', num_epochs_nade_p, p1_nade_model, p1_nade_optimizer, training_loader)
```

```
P1_NADE Epoch 153 Loss: 0.1247
```

```
P1_NADE Epoch 153 Loss: 0.1247
P1_NADE Epoch 154 Loss: 0.1244
P1_NADE Epoch 155 Loss: 0.1241
P1_NADE Epoch 156 Loss: 0.1237
P1_NADE Epoch 157 Loss: 0.1234
P1_NADE Epoch 158 Loss: 0.1231
P1_NADE Epoch 159 Loss: 0.1228
P1_NADE Epoch 160 Loss: 0.1225
P1_NADE Epoch 161 Loss: 0.1222
P1_NADE Epoch 162 Loss: 0.1219
P1_NADE Epoch 163 Loss: 0.1216
P1_NADE Epoch 164 Loss: 0.1213
P1_NADE Epoch 165 Loss: 0.1210
P1_NADE Epoch 166 Loss: 0.1207
P1_NADE Epoch 167 Loss: 0.1204
P1_NADE Epoch 168 Loss: 0.1201
P1_NADE Epoch 169 Loss: 0.1199
P1_NADE Epoch 170 Loss: 0.1196
P1_NADE Epoch 171 Loss: 0.1193
P1_NADE Epoch 172 Loss: 0.1190
P1_NADE Epoch 173 Loss: 0.1188
P1_NADE Epoch 174 Loss: 0.1185
P1_NADE Epoch 175 Loss: 0.1182
P1_NADE Epoch 176 Loss: 0.1180
P1_NADE Epoch 177 Loss: 0.1177
P1_NADE Epoch 178 Loss: 0.1175
P1_NADE Epoch 179 Loss: 0.1172
P1_NADE Epoch 180 Loss: 0.1170
P1_NADE Epoch 181 Loss: 0.1167
P1_NADE Epoch 182 Loss: 0.1165
P1_NADE Epoch 183 Loss: 0.1162
P1_NADE Epoch 184 Loss: 0.1160
P1_NADE Epoch 185 Loss: 0.1157
P1_NADE Epoch 186 Loss: 0.1155
P1_NADE Epoch 187 Loss: 0.1153
P1_NADE Epoch 188 Loss: 0.1150
P1_NADE Epoch 189 Loss: 0.1148
P1_NADE Epoch 190 Loss: 0.1146
P1_NADE Epoch 191 Loss: 0.1144
P1_NADE Epoch 192 Loss: 0.1141
P1_NADE Epoch 193 Loss: 0.1139
P1_NADE Epoch 194 Loss: 0.1137
P1_NADE Epoch 195 Loss: 0.1135
P1_NADE Epoch 196 Loss: 0.1133
P1_NADE Epoch 197 Loss: 0.1131
P1_NADE Epoch 198 Loss: 0.1128
P1_NADE Epoch 199 Loss: 0.1126
```

```
# p2: reverse permutation (from end to beginning)
p2 = list(reversed(range(D)))
p2_nade_model = NADE(input_size=D, internal_size=512, permutation=p2).to("cuda")

p2_nade_optimizer = torch.optim.Adam(p2_nade_model.parameters(), lr=lr)

train_models('P2_NADE', num_epochs_nade_p, p2_nade_model, p2_nade_optimizer, training_loader)
```

```
P2_NADE Epoch 160 Loss: 0.1212
```

```
P2_NADE Epoch 169 Loss: 0.1212
P2_NADE Epoch 170 Loss: 0.1210
P2_NADE Epoch 171 Loss: 0.1207
P2_NADE Epoch 172 Loss: 0.1204
P2_NADE Epoch 173 Loss: 0.1202
P2_NADE Epoch 174 Loss: 0.1199
P2_NADE Epoch 175 Loss: 0.1197
P2_NADE Epoch 176 Loss: 0.1194
P2_NADE Epoch 177 Loss: 0.1192
P2_NADE Epoch 178 Loss: 0.1189
P2_NADE Epoch 179 Loss: 0.1187
P2_NADE Epoch 180 Loss: 0.1184
P2_NADE Epoch 181 Loss: 0.1182
P2_NADE Epoch 182 Loss: 0.1179
P2_NADE Epoch 183 Loss: 0.1177
P2_NADE Epoch 184 Loss: 0.1174
P2_NADE Epoch 185 Loss: 0.1172
P2_NADE Epoch 186 Loss: 0.1170
P2_NADE Epoch 187 Loss: 0.1167
P2_NADE Epoch 188 Loss: 0.1165
P2_NADE Epoch 189 Loss: 0.1163
P2_NADE Epoch 190 Loss: 0.1161
P2_NADE Epoch 191 Loss: 0.1158
P2_NADE Epoch 192 Loss: 0.1156
P2_NADE Epoch 193 Loss: 0.1154
P2_NADE Epoch 194 Loss: 0.1152
P2_NADE Epoch 195 Loss: 0.1150
P2_NADE Epoch 196 Loss: 0.1148
P2_NADE Epoch 197 Loss: 0.1145
P2_NADE Epoch 198 Loss: 0.1143
P2_NADE Epoch 199 Loss: 0.1141
```

```
# p3: custom permutation starting from row 13, then 14, then 12, then 15, ...
rows = []
for offset in range(28):
    if offset % 2 == 0:
        rows.append(13 + offset//2)
    else:
        rows.append(13 - (offset+1)//2)
p3 = [r*28 + c for c in range(28) for r in rows]
p3_nade_model = NADE(input_size=D, internal_size=512, permutation=p3).to("cuda")

p3_nade_optimizer = torch.optim.Adam(p3_nade_model.parameters(), lr=lr)

train_models('P3_NADE', num_epochs_nade_p, p3_nade_model, p3_nade_optimizer, training_loader)
```

```
P3_NADE Epoch 170 Loss: 0.1161
```

```
P3_NADE Epoch 179 Loss: 0.1161
P3_NADE Epoch 180 Loss: 0.1159
P3_NADE Epoch 181 Loss: 0.1157
P3_NADE Epoch 182 Loss: 0.1154
P3_NADE Epoch 183 Loss: 0.1152
P3_NADE Epoch 184 Loss: 0.1150
P3_NADE Epoch 185 Loss: 0.1147
P3_NADE Epoch 186 Loss: 0.1145
P3_NADE Epoch 187 Loss: 0.1143
P3_NADE Epoch 188 Loss: 0.1140
P3_NADE Epoch 189 Loss: 0.1138
P3_NADE Epoch 190 Loss: 0.1136
P3_NADE Epoch 191 Loss: 0.1134
P3_NADE Epoch 192 Loss: 0.1132
P3_NADE Epoch 193 Loss: 0.1130
P3_NADE Epoch 194 Loss: 0.1127
P3_NADE Epoch 195 Loss: 0.1125
P3_NADE Epoch 196 Loss: 0.1123
P3_NADE Epoch 197 Loss: 0.1121
P3_NADE Epoch 198 Loss: 0.1119
P3_NADE Epoch 199 Loss: 0.1117
```

```
# p4: custom permutation starting from column 13, then 14, then 12, then 15, ...
cols = []
for offset in range(28):
    if offset % 2 == 0:
        cols.append(13 + offset//2)
    else:
        cols.append(13 - (offset+1)//2)
p4 = [r*28 + c for r in range(28) for c in cols]
p4_nade_model = NADE(input_size=D, internal_size=512, permutation=p4).to("cuda")

p4_nade_optimizer = torch.optim.Adam(p4_nade_model.parameters(), lr=lr)

train_models('P4_NADE', num_epochs_nade_p, p4_nade_model, p4_nade_optimizer, training_loader)
```

```
P4_NADE Epoch 000 Loss: 0.5934
P4_NADE Epoch 001 Loss: 0.5040
P4_NADE Epoch 002 Loss: 0.4596
P4_NADE Epoch 003 Loss: 0.4332
P4_NADE Epoch 004 Loss: 0.4141
P4_NADE Epoch 005 Loss: 0.3987
P4_NADE Epoch 006 Loss: 0.3854
P4_NADE Epoch 007 Loss: 0.3735
P4_NADE Epoch 008 Loss: 0.3626
P4_NADE Epoch 009 Loss: 0.3526
P4_NADE Epoch 010 Loss: 0.3432
P4_NADE Epoch 011 Loss: 0.3346
P4_NADE Epoch 012 Loss: 0.3265
P4_NADE Epoch 013 Loss: 0.3190
P4_NADE Epoch 014 Loss: 0.3121
P4_NADE Epoch 015 Loss: 0.3056
P4_NADE Epoch 016 Loss: 0.2995
P4_NADE Epoch 017 Loss: 0.2939
P4_NADE Epoch 018 Loss: 0.2886
P4_NADE Epoch 019 Loss: 0.2837
P4_NADE Epoch 020 Loss: 0.2790
P4_NADE Epoch 021 Loss: 0.2746
P4_NADE Epoch 022 Loss: 0.2705
P4_NADE Epoch 023 Loss: 0.2666
P4_NADE Epoch 024 Loss: 0.2629
P4_NADE Epoch 025 Loss: 0.2593
P4_NADE Epoch 026 Loss: 0.2559
P4_NADE Epoch 027 Loss: 0.2527
P4_NADE Epoch 028 Loss: 0.2496
P4_NADE Epoch 029 Loss: 0.2466
P4_NADE Epoch 030 Loss: 0.2438
P4_NADE Epoch 031 Loss: 0.2410
P4_NADE Epoch 032 Loss: 0.2383
P4_NADE Epoch 033 Loss: 0.2358
P4_NADE Epoch 034 Loss: 0.2333
P4_NADE Epoch 035 Loss: 0.2309
P4_NADE Epoch 036 Loss: 0.2285
P4_NADE Epoch 037 Loss: 0.2263
P4_NADE Epoch 038 Loss: 0.2241
P4_NADE Epoch 039 Loss: 0.2220
P4_NADE Epoch 040 Loss: 0.2199
P4_NADE Epoch 041 Loss: 0.2179
P4_NADE Epoch 042 Loss: 0.2160
P4_NADE Epoch 043 Loss: 0.2141
P4_NADE Epoch 044 Loss: 0.2122
P4_NADE Epoch 045 Loss: 0.2104
P4_NADE Epoch 046 Loss: 0.2087
```

```
P4_NADE Epoch 047 Loss: 0.2070
P4_NADE Epoch 048 Loss: 0.2053
P4_NADE Epoch 049 Loss: 0.2036
P4_NADE Epoch 050 Loss: 0.2020
P4_NADE Epoch 051 Loss: 0.2005
P4_NADE Epoch 052 Loss: 0.1989
P4_NADE Epoch 053 Loss: 0.1974
P4_NADE Epoch 054 Loss: 0.1959
P4_NADE Epoch 055 Loss: 0.1945
P4_NADE Epoch 056 Loss: 0.1931
P4_NADE Epoch 057 Loss: 0.1917
```

```
# p5: concentric circles (spiral) permutation for MNIST 28x28 images
def spiral_permutation(n):
    perm = []
    left, right, top, bottom = 0, n-1, 0, n-1
    while left <= right and top <= bottom:
        # Top row (left to right)
        for c in range(left, right+1):
            perm.append(top*n + c)
        top += 1
        # Right column (top to bottom)
        for r in range(top, bottom+1):
            perm.append(r*n + right)
        right -= 1
        # Bottom row (right to left)
        if top <= bottom:
            for c in range(right, left-1, -1):
                perm.append(bottom*n + c)
            bottom -= 1
        # Left column (bottom to top)
        if left <= right:
            for r in range(bottom, top-1, -1):
                perm.append(r*n + left)
            left += 1
    return perm
p5 = spiral_permutation(28)
p5_nade_model = NADE(input_size=D, internal_size=512, permutation=p5).to("cuda")


p5_nade_optimizer = torch.optim.Adam(p5_nade_model.parameters(), lr=lr)

train_models('P5_NADE', num_epochs_nade_p, p5_nade_model, p5_nade_optimizer, training_loader)
```

```
P5_NADE Epoch 149 Loss: 0.1223
```

```
P5_NADE Epoch 150 Loss: 0.1220
P5_NADE Epoch 151 Loss: 0.1218
P5_NADE Epoch 152 Loss: 0.1216
P5_NADE Epoch 153 Loss: 0.1213
P5_NADE Epoch 154 Loss: 0.1211
P5_NADE Epoch 155 Loss: 0.1209
P5_NADE Epoch 156 Loss: 0.1206
P5_NADE Epoch 157 Loss: 0.1204
P5_NADE Epoch 158 Loss: 0.1202
P5_NADE Epoch 159 Loss: 0.1200
P5_NADE Epoch 160 Loss: 0.1198
P5_NADE Epoch 161 Loss: 0.1196
P5_NADE Epoch 162 Loss: 0.1193
P5_NADE Epoch 163 Loss: 0.1191
P5_NADE Epoch 164 Loss: 0.1189
P5_NADE Epoch 165 Loss: 0.1187
P5_NADE Epoch 166 Loss: 0.1185
P5_NADE Epoch 167 Loss: 0.1183
P5_NADE Epoch 168 Loss: 0.1181
P5_NADE Epoch 169 Loss: 0.1179
P5_NADE Epoch 170 Loss: 0.1177
```

```
with torch.no_grad():
    # Load all permuted NADE models
    nade_models = []
    model_names = ['P1_NADE', 'P2_NADE', 'P3_NADE', 'P4_NADE', 'P5_NADE']
    for name in model_names:
        model = torch.load(f'{name}.model')
        model.eval()
        nade_models.append(model)

    # Generate 8 samples for each model
    num_samples = 8
    fig, axes = plt.subplots(len(nade_models), num_samples, figsize=(num_samples*2, len(nade_models)*2))
    for i, model in enumerate(nade_models):
        samples = model.sample(num_samples)
        samples = samples.cpu().numpy().reshape(-1, 28, 28)
        for j in range(num_samples):
            axes[i, j].imshow(samples[j], cmap='gray')
            axes[i, j].axis('off')
            if i == 0:
                axes[i, j].set_title(f"Sample {j+1}")
        axes[i, 0].set_ylabel(model_names[i], rotation=0, labelpad=40, fontsize=12)

    plt.tight_layout()
    plt.show()

    # Calculate p(x) for the test dataset for each model
    px_results = {}
    for i, model in enumerate(nade_models):
        px_list = []
        for batch in testing_loader:
            x, _ = batch
            x = x.view(x.size(0), -1).to('cuda')
            px, _ = model.evalP(x)
            px_list.append(px.cpu())
        px_all = torch.cat(px_list)
        px_results[model_names[i]] = px_all

    # Print mean log p(x) for each model
    for name in model_names:
        mean_log_px = torch.log(px_results[name] + 1e-8).mean().item()
        print(f"{name}: Mean log p(x) on test set = {mean_log_px:.4f}")
```

```
<ipython-input-37-16fe76ab375d>:6: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), whic
  model = torch.load(f'{name}.model')
```